

PFAM

Complete A–Z Implementation Plan

Solo Builder Edition · Full SRS · 5-Tier Attribution · Cursor-Ready

Stack	Stack B — Next.js (frontend) + FastAPI/Python (backend)
Database	Neon (serverless Postgres)
Auth	Clerk
Workers	Celery + Upstash Redis on Railway
Total Phases	13 phases, ~6–9 months solo at 4–6 hrs/day
How to use this doc	Follow manually steps yourself. Paste Cursor prompts into Cursor Agent mode. Tick every checkpoint before moving to next phase.

How to Use This Document

Element	Meaning
 YOU DO THIS MANUALLY	Steps you must perform yourself — account setup, API approvals, config. No AI can do these.
 CURSOR PROMPT	Paste this verbatim into Cursor Agent mode. Read the output before accepting.
 PHASE COMPLETE WHEN...	Every checkbox must pass before starting the next phase. Non-negotiable.
 Warning	A known trap that will cost you hours if ignored.
 Tip	A shortcut or best practice worth knowing.

 Open Cursor. Set model to Claude Sonnet. Enable Agent mode. Keep this document open beside it. Work phase by phase. Never skip a checkpoint.

Step 0 — Create Your .cursorrules File

Create a file named `.cursorrules` in the root of BOTH repos (pfam-frontend and pfam-backend). Paste the contents below. Cursor reads this on every single prompt — it is the most important thing you can do for code quality and consistency.

```
# PFAM — Cursor Rules
# Profit-First Ad Manager | Solo Build | Full SRS Implementation

## Project
Building a multi-tenant SaaS called PFAM (Profit-First Ad Manager).
D2C e-commerce brands connect Shopify + Meta Ads + Google Ads.
System calculates true net profit per campaign and automates pausing unprofitable ones.

## Stack
Frontend: Next.js 14 App Router + TypeScript + Tailwind + shadcn/ui → deployed on Vercel
Backend: Python 3.12 + FastAPI + SQLAlchemy 2.0 + Alembic → deployed on Railway
Database: Neon (serverless PostgreSQL 15)
Auth: Clerk (JWT verified in FastAPI via JWKS endpoint)
Workers: Celery + Upstash Redis (broker)
Storage: Cloudflare R2 (S3-compatible)
Email: Resend
Payments: Stripe

## Core Profit Formula (NEVER deviate from this)
Net Profit = Attributed Revenue - Ad Spend - Attributed COGS - Estimated Returns - Platform Fees
All monetary values stored as INTEGER CENTS. Never floats for money.
Net Profit % = (Net Profit / Attributed Revenue) * 100
True ROAS = Attributed Revenue / Ad Spend

## 5-Tier Attribution Engine
Tier 1: Direct click ID match (fbclid/gclid in Shopify UTM params) — confidence 0.95
Tier 2: Conversion ID match (Meta Pixel / Google Tag event metadata) — confidence 0.85
Tier 3: SKU-weighted — distribute revenue proportional to campaign spend on matching SKUs — confidence 0.70
Tier 4: Blended — proportional across all active campaigns by spend volume — confidence 0.50
Tier 5: ML (XGBoost classifier trained on Tier1/2 ground truth) — confidence 0.60-0.85

## Multi-Tenancy Rules (CRITICAL — never break these)
Every DB table has org_id UUID column.
Every query MUST include WHERE org_id = :current_org_id.
Supabase RLS equivalent: SQLAlchemy always filters by org_id at service layer.
Never write a query that can return data from multiple organizations.

## Money Rules
Always store money as integer cents (e.g., $10.50 = 1050).
Always divide by 100 only at display layer.
Use Python Decimal for intermediate calculations, not float.
Never use float arithmetic on monetary values.
```

```
## Code Conventions
Python: snake_case, type hints on all functions, Pydantic v2 for request/response
models.
TypeScript: camelCase, strict mode, no any types.
All API responses: { data: ..., error: null } or { data: null, error: { code,
message } }
All timestamps: UTC ISO 8601 (e.g., 2026-02-13T10:30:00Z)
All IDs: UUID v4

## Security Rules
Never log OAuth tokens, API keys, or PII.
All platform tokens (Shopify, Meta, Google) stored AES-256 encrypted.
Idempotency: every sync job must be safe to run twice (upsert, not insert).
Audit log is append-only – never add UPDATE or DELETE to audit_log table.

## Error Handling
All external API calls (Shopify, Meta, Google) wrapped in try/except with specific
error types.
Failed sync jobs: retry 3 times with exponential backoff (5min, 15min, 60min).
Platform API rate limits: raise specific RateLimitError, not generic Exception.
Never swallow exceptions silently – always log with context.

## Testing
After building each engine (attribution, profit, rules), write tests matching the
SRS test scenarios.
Profit engine must pass: calculated value within ±1% of manually computed ground
truth.
Attribution Tier 1+2: must correctly attribute ≥92% of orders with known click
IDs.
```

 Do not skip this step. Every Cursor prompt in this document assumes these rules are active. Without `.cursorrules`, the AI will invent its own conventions and your codebase will become inconsistent by Phase 3.

PHASE 0 Accounts, Credits & Day-1 Setup **1–2 days**

Before writing a single line of code, complete all manual account setup. Some of these (Meta App Review) take weeks — start them NOW so they are approved by the time you need them.

 YOU DO THIS MANUALLY — Create all accounts and claim student credits

1. GitHub: Create account + organisation (e.g., pfam-app). Apply for Student Developer Pack at education.github.com with .edu email.
2. Vercel: Sign up at vercel.com → connect GitHub → redeem Student Pack for Pro.
3. Neon: Sign up at neon.tech → create project "pfam-prod" → create branches: main, dev, staging.
4. Clerk: Sign up at clerk.com → create application "PFAM" → enable Google OAuth → copy PUBLISHABLE_KEY and SECRET_KEY.
5. Railway: Sign up at railway.app → connect GitHub → claim Student Pack extra credits.
6. Upstash: Sign up at upstash.com → create Redis database "pfam-redis" → copy REST URL and token.
7. Cloudflare: Sign up at cloudflare.com → create R2 bucket "pfam-storage" → create API token.
8. Resend: Sign up at resend.com → add your domain → verify DNS → copy API key.
9. Sentry: Sign up at sentry.io → redeem GitHub Student Pack → create two projects: pfam-frontend, pfam-backend.
10. Stripe: Sign up at stripe.com → stay in TEST mode → create products for each pricing tier → copy keys.
11. PostHog: Sign up at posthog.com → create project → copy API key.

 YOU DO THIS MANUALLY — Platform API accounts — START THESE NOW (approval takes weeks)

1. Shopify Partners: Go to partners.shopify.com → create partner account → create a new app → set redirect URL to <http://localhost:3000/api/connect/shopify/callback> → copy API key and secret.
2. Meta for Developers: Go to developers.facebook.com → create app → choose Business type → add Marketing API product → apply for ads_read and ads_management permissions. For ads_management (needed for automated pauses) submit App Review with a Loom screencast of your prototype. This takes 2–6 weeks. Start immediately.
3. Google Ads API: Go to ads.google.com/aw/apidevelopers → apply for developer token → Basic Access is auto-approved (15K ops/day). Create OAuth2 credentials at console.cloud.google.com → APIs & Services → Credentials.
4. Create a .env.local (frontend) and .env (backend) file. Never commit these to Git. Add all keys you have so far — leave placeholders for the ones still being approved.

 PHASE COMPLETE WHEN...

- ☐ GitHub Student Pack approved and all tools redeemed.
- ☐ Neon project created with main/dev/staging branches. Connection string saved.
- ☐ Clerk app created. Both keys saved in .env files.
- ☐ Railway project created. Upstash Redis connected.
- ☐ Shopify Partner app created. API credentials saved.
- ☐ Meta app submitted for API access review.
- ☐ Google developer token received (Basic Access).
- ☐ All .env files populated with placeholders for every key needed.

PHASE 1 Project Scaffolding & Repo Setup **2–3 days** **YOU DO THIS MANUALLY** — Create both repos

1. Create pfam-frontend repo in your GitHub org. Clone locally.
2. Create pfam-backend repo in your GitHub org. Clone locally.
3. In pfam-frontend: run `→ npx create-next-app@latest . --typescript --tailwind --eslint --app --src-dir`
4. In pfam-frontend: run `→ npx shadcn-ui@latest init` (choose Default style, Slate base colour, yes CSS variables)
5. In pfam-backend: run `→ python3 -m venv venv && source venv/bin/activate`
6. In pfam-backend: run `→ pip install fastapi uvicorn sqlalchemy alembic pycopg2-binary celery redis pydantic python-jose cryptography httpx pandas polars python-dotenv sentry-sdk`
7. Create requirements.txt: run `→ pip freeze > requirements.txt`
8. Create .cursorrules in BOTH repo roots (paste content from Step 0 above).
9. Create .env.local (frontend) and .env (backend) with all keys from Phase 0.

 CURSOR PROMPT — Scaffold FastAPI backend structure

Set up the complete folder structure for the PFAM FastAPI backend as specified in .cursorrules.

Create these folders and files:

```

app/__init__.py
app/main.py (FastAPI app, CORS, Sentry, router registration)
app/db.py (SQLAlchemy async engine for Neon, session factory)
app/config.py (Pydantic BaseSettings reading from .env)
app/middleware/auth.py (Clerk JWT verification using JWKS)
app/routers/__init__.py
app/models/__init__.py
app/services/__init__.py
app/workers/__init__.py
celery_app.py (Celery config using Upstash Redis as broker)
alembic.ini (Alembic config pointing to Neon DATABASE_URL)
alembic/env.py (async Alembic setup)
Procfile (web: uvicorn app.main:app | worker: celery -A celery_app worker |
beat: celery -A celery_app beat)
```

In app/main.py: register health check GET /health returning {status: ok, version: 1.0}.

In app/middleware/auth.py: fetch Clerk JWKS from `https://api.clerk.dev/v1/jwks`, verify JWT signature, extract org_id and user_id, inject into request.state. Return 401 if token invalid or missing.

In app/db.py: create async SQLAlchemy engine for Neon using DATABASE_URL from config.

Follow all conventions in .cursorrules exactly.

 CURSOR PROMPT — Scaffold Next.js frontend structure

Set up the complete folder structure for the PFAM Next.js frontend.

Install: `npm install @clerk/nextjs @tanstack/react-query axios recharts lucide-react`

Create these files:

```
src/app/layout.tsx (ClerkProvider wrapping everything, QueryClientProvider)
src/app/(auth)/sign-in/page.tsx (Clerk SignIn component)
src/app/(auth)/sign-up/page.tsx (Clerk SignUp component)
src/app/(dashboard)/layout.tsx (sidebar nav + topbar, protected by Clerk
middleware)
src/app/(dashboard)/overview/page.tsx (empty shell with "Dashboard coming
soon")
src/middleware.ts (Clerk middleware protecting all /dashboard routes)
src/lib/api.ts (Axios instance pointing to NEXT_PUBLIC_API_URL, attaches Clerk
JWT)
src/lib/types.ts (shared TypeScript types matching backend Pydantic models)
```

The api.ts Axios instance must: get token from Clerk useAuth(),
attach as Authorization: Bearer <token> on every request.

Follow all conventions in .cursorsrules.

☑ PHASE COMPLETE WHEN...

- ☐ GET /health on the FastAPI backend returns 200 {"status": "ok"}.
- ☐ Next.js dev server starts without errors.
- ☐ Clerk sign-up and sign-in pages render at /sign-in and /sign-up.
- ☐ Navigating to /overview redirects to /sign-in if not logged in.
- ☐ After signing in, /overview loads (even if empty).
- ☐ Both .cursorsrules files are in place.

PHASE 2 Database Schema & Migrations

🕒 3–4 days

The schema is the foundation. Get this right before building anything else. Every mistake here compounds later.

⚠️ Store ALL money as integer cents. If Cursor ever writes a NUMERIC or FLOAT column for a monetary value, reject it and ask for INTEGER.

CURSOR PROMPT — Create complete database schema — Part 1 (Core entities)

Create SQLAlchemy models and Alembic migration for these tables.
Every table MUST have: id UUID PK default gen_random_uuid(), org_id UUID NOT NULL, created_at TIMESTAMPTZ default now(), updated_at TIMESTAMPTZ.
ALL monetary values as INTEGER (cents). No floats. No numerics for money.

```
TABLE: organizations
  name TEXT, billing_plan TEXT, stripe_customer_id TEXT, stripe_subscription_id
  TEXT,
  base_currency TEXT default USD, data_region TEXT default us-east-1
```

```
TABLE: users
  clerk_user_id TEXT UNIQUE NOT NULL, email TEXT, name TEXT,
  role TEXT CHECK(role IN (owner,admin,analyst,read_only)) default read_only
```

```
TABLE: stores (Shopify)
  shopify_store_id TEXT NOT NULL, shop_domain TEXT NOT NULL,
  access_token_enc TEXT, access_token_iv TEXT,
  last_sync_at TIMESTAMPTZ, sync_status TEXT, total_orders_synced INTEGER default
  0
```

```
TABLE: ad_accounts (Meta/Google/TikTok)
  platform TEXT CHECK(platform IN (meta,google,tiktok)) NOT NULL,
  platform_account_id TEXT NOT NULL, account_name TEXT,
  access_token_enc TEXT, access_token_iv TEXT, refresh_token_enc TEXT,
  token_expires_at TIMESTAMPTZ, currency TEXT default USD,
  last_sync_at TIMESTAMPTZ, sync_status TEXT
```

Create alembic migration. Run: alembic revision --autogenerate -m "core_entities"
Apply: alembic upgrade head
Verify tables exist in Neon dashboard.

CURSOR PROMPT — Create database schema — Part 2 (Ad & Order data)

Continue building SQLAlchemy models and migration for ad and order data tables.

```
TABLE: campaigns
  ad_account_id UUID FK, platform_campaign_id TEXT, name TEXT,
  status TEXT, objective TEXT, daily_budget_cents INTEGER, start_date DATE,
  end_date DATE
```

```
TABLE: ad_sets
  campaign_id UUID FK, platform_adset_id TEXT, name TEXT,
```



```

status TEXT, daily_budget_cents INTEGER, targeting_summary JSONB

TABLE: ad_insights (daily metrics)
  ad_set_id UUID FK, date DATE NOT NULL,
  spend_cents INTEGER, impressions INTEGER, clicks INTEGER,
  reach INTEGER, conversions INTEGER, conversion_value_cents INTEGER,
  cpm_cents INTEGER, cpc_cents INTEGER, ctr NUMERIC(8,4)
  UNIQUE(ad_set_id, date)

TABLE: orders (from Shopify)
  store_id UUID FK, shopify_order_id TEXT NOT NULL,
  ordered_at TIMESTAMPTZ, total_amount_cents INTEGER, total_discounts_cents
INTEGER,
  currency TEXT, customer_id TEXT, financial_status TEXT, fulfillment_status TEXT
  UNIQUE(store_id, shopify_order_id)

TABLE: line_items
  order_id UUID FK, shopify_line_item_id TEXT,
  product_id TEXT, variant_id TEXT, sku TEXT,
  quantity INTEGER, unit_price_cents INTEGER,
  unit_cogs_cents INTEGER, unit_cogs_source TEXT CHECK(IN
(shopify,csv>manual,estimated))

TABLE: returns
  order_id UUID FK, line_item_id UUID FK nullable,
  refund_amount_cents INTEGER, quantity_returned INTEGER,
  reason TEXT, reason_category TEXT CHECK(IN
(defective>wrong>change_mind>sizing>other)),
  refunded_at TIMESTAMPTZ

Create and apply migration.

```

CURSOR PROMPT — Create database schema — Part 3 (Attribution, Profit, Rules)

Create remaining tables for the attribution engine, profit metrics, and automation.

```

TABLE: attributed_orders
  order_id UUID FK, ad_set_id UUID FK,
  attribution_tier INTEGER CHECK(BETWEEN 1 AND 5),
  confidence_score NUMERIC(4,3), attribution_method TEXT,
  matched_click_id TEXT, attributed_revenue_cents INTEGER,
  window_start TIMESTAMPTZ, window_end TIMESTAMPTZ
  UNIQUE(order_id, ad_set_id)

TABLE: sku_return_rates
  sku TEXT NOT NULL, trailing_90d_rate NUMERIC(6,4), trailing_180d_rate
NUMERIC(6,4),
  manual_override_rate NUMERIC(6,4), last_computed_at TIMESTAMPTZ
  UNIQUE(org_id, sku)

TABLE: cogs_settings
  scope TEXT CHECK(IN (sku>category>global)), scope_value TEXT,
  cogs_type TEXT CHECK(IN (absolute>percentage)),
  cogs_value_cents INTEGER, currency TEXT, source TEXT CHECK(IN
(shopify>csv>>manual))

```

```

TABLE: profit_metrics (computed snapshots)
  ad_set_id UUID FK, window_type TEXT CHECK(IN (daily,7d,14d,30d)),
  window_start TIMESTAMPTZ, window_end TIMESTAMPTZ,
  spend_cents INTEGER, attributed_revenue_cents INTEGER,
  attributed_cogs_cents INTEGER, estimated_returns_cents INTEGER,
  platform_fees_cents INTEGER, net_profit_cents INTEGER,
  net_profit_pct NUMERIC(8,4), true_roas NUMERIC(8,4),
  order_count INTEGER, attribution_coverage_pct NUMERIC(6,4), computed_at
TIMESTAMPTZ
  UNIQUE(ad_set_id, window_type, window_start, window_end)

TABLE: automation_rules
  name TEXT, platform TEXT, scope TEXT CHECK(IN (account,campaign,ad_set,ad)),
  conditions JSONB NOT NULL, action_type TEXT, action_params JSONB,
  guardrails JSONB, is_active BOOLEAN default false,
  created_by UUID FK users

TABLE: rule_executions
  rule_id UUID FK, evaluated_at TIMESTAMPTZ,
  entities_evaluated INTEGER, entities_triggered INTEGER,
  entities_blocked_by_guardrail INTEGER, action_queued BOOLEAN

TABLE: audit_log (APPEND ONLY — no updates ever)
  actor_user_id UUID nullable, actor_type TEXT CHECK(IN (user,system)),
  action_type TEXT, entity_type TEXT, entity_id TEXT, platform TEXT,
  rule_id UUID nullable, metric_snapshot JSONB,
  api_request JSONB, api_response_code INTEGER, api_response JSONB
-- NOTE: NO updated_at column. Audit log is immutable.

TABLE: notifications
  trigger_type TEXT, channel TEXT CHECK(IN (email,slack,webhook)),
  payload JSONB, delivered_at TIMESTAMPTZ, delivery_status TEXT, retry_count
INTEGER

Create indexes:
  profit_metrics(ad_set_id, window_type, window_start, window_end)
  attributed_orders(ad_set_id, window_start, window_end)
  orders(store_id, ordered_at)
  ad_insights(ad_set_id, date)
  audit_log(org_id, created_at)

Create and apply migration.

```

☒ PHASE COMPLETE WHEN...

- ☐ alembic upgrade head runs without errors.
- ☐ All 17 tables visible in Neon dashboard.
- ☐ Every table has org_id column.
- ☐ audit_log table has NO updated_at column.
- ☐ profit_metrics, orders, ad_insights all have UNIQUE constraints.
- ☐ All money columns are INTEGER type — verify in Neon schema view.

PHASE 3 Shopify Connector — OAuth + Data Ingestion

🕒 1.5–2 weeks

🔑 YOU DO THIS MANUALLY — Shopify setup before coding

1. In your Shopify Partner dashboard: set App URL to `http://localhost:8000` and Allowed redirect URLs to `http://localhost:8000/api/connect/shopify/callback`.
2. Install a Shopify development store from your Partner dashboard to test with.
3. Note your Shopify App API key and secret — add to backend `.env` as `SHOPIFY_API_KEY` and `SHOPIFY_API_SECRET`.

🔗 CURSOR PROMPT — Shopify OAuth flow

Build the Shopify OAuth connector in `app/routers/connectors.py` and `app/services/connectors/shopify.py`.

Step 1 – Initiate OAuth:

```
GET /api/connect/shopify?shop={shop_domain}
Validate shop domain format (*.myshopify.com).
Generate nonce, store in Redis with 10-min TTL.
Redirect to: https://{shop}/admin/oauth/authorize
?client_id={SHOPIFY_API_KEY}
&scope=read_orders,read_products,read_inventory,read_reports,read_fulfillments
&redirect_uri={REDIRECT_URI}
&state={nonce}
```

Step 2 – OAuth callback:

```
GET /api/connect/shopify/callback?code=...&shop=...&state=...
Verify state matches nonce in Redis (prevent CSRF).
Exchange code for access token via POST https://{shop}/admin/oauth/access_token.
Encrypt access token with AES-256 using ENCRYPTION_KEY from env.
Store encrypted token + IV separately in stores table.
Save store record in DB (org_id from JWT, shopify_store_id, shop_domain).
Trigger background job: sync_shopify_historical.delay(store_id, days=90)
Return redirect to frontend /settings?connected=shopify
```

Encryption helper: create `app/utils/encryption.py`

```
encrypt(plaintext: str) -> tuple[str, str] # returns (ciphertext_b64, iv_b64)
decrypt(ciphertext_b64: str, iv_b64: str) -> str
Use AES-256-CBC with key from ENCRYPTION_KEY env var.
```

Follow all `.cursorrules` conventions.

🔗 CURSOR PROMPT — Shopify order ingestion worker

Build `app/workers/sync_shopify.py` – Celery task for ingesting Shopify orders.

```
@celery_app.task(bind=True, max_retries=3)
def sync_shopify_orders(self, store_id: str, since_date: str = None):
    Load store from DB. Decrypt access token.
    If since_date is None, use 90 days ago.
    Call Shopify Orders API: GET /admin/api/2024-01/orders.json
    with params: status=any, created_at_min={since_date}, limit=250
    Handle pagination: loop while next page link exists in Link header.
    For each order:
```

```
Upsert order record (UNIQUE on store_id + shopify_order_id).
Upsert each line_item (quantity, unit_price_cents, sku, variant_id).
Set unit_cogs_cents from line_item.cost_per_item if available, else NULL.
Set unit_cogs_source = shopify if cost available, else estimated.
Handle rate limits: if 429 received, wait rate_limit_delay from headers, then
retry.
Handle errors: on Shopify API 5xx, raise self.retry(countdown=300, exc=exc).
Update stores.last_sync_at and stores.sync_status on completion.
Log: orders synced count, errors, duration.
```

IMPORTANT: This job must be idempotent.

Running it twice for the same date range must NOT create duplicate records.

Use `INSERT ... ON CONFLICT (store_id, shopify_order_id) DO UPDATE.`

Also create `sync_shopify_refunds(store_id, since_date)` task:

Fetch refunds from `GET /admin/api/2024-01/orders.json?status=any` (refunds are nested in orders)

Upsert returns table for each refund line item.

CURSOR PROMPT — Shopify API endpoint for frontend

Create `app/routers/stores.py`:

`GET /api/stores` — list all connected stores for org

`GET /api/stores/{store_id}` — store details including sync status, last sync, order count

`POST /api/stores/{store_id}/sync` — trigger manual sync (rate-limit: once per 5 min per store)

Check Redis for rate limit key before triggering.

`DELETE /api/stores/{store_id}` — disconnect store (delete access token, mark disconnected)

All endpoints must:

Verify JWT via auth middleware (`org_id` in `request.state`).

Filter all DB queries by `org_id`.

Return 404 if store not found OR belongs to different org.

PHASE COMPLETE WHEN...

- ☐ OAuth flow completes — store record created in DB with encrypted token.
- ☐ `sync_shopify_orders` task runs and imports 90 days of orders.
- ☐ Running the same sync task twice creates zero duplicates (test this).
- ☐ Refunds are imported and visible in returns table.
- ☐ `GET /api/stores` returns connected stores for the org.
- ☐ `unit_cogs_source` = "shopify" for products with cost data, "estimated" for others.

PHASE 4 Meta & Google Ads Connectors

🕒 2–3 weeks

⚠️ Meta App Review for ads_management (automated pausing) may still be in progress. Build everything using test ad accounts. Automated action execution (Phase 7) comes later — for now just build the data ingestion.

🔑 YOU DO THIS MANUALLY — Meta setup

1. In Meta for Developers: add `http://localhost:8000/api/connect/meta/callback` as a valid OAuth redirect URI.
2. Get your App ID and App Secret. Add to `.env` as `META_APP_ID` and `META_APP_SECRET`.
3. Create a test ad account in Meta Business Manager for development testing.
4. Get a System User access token for your test account to use during development (avoids user re-auth constantly).

🔗 CURSOR PROMPT — Meta Ads OAuth + data ingestion

Build `app/services/connectors/meta.py` and `app/workers/sync_meta.py`.

--- OAuth Flow ---

GET `/api/connect/meta:`

Redirect to: `https://www.facebook.com/v18.0/dialog/oauth?client_id={META_APP_ID}&redirect_uri={REDIRECT_URI}&scope=ads_read,ads_management&state={nonce}`

GET `/api/connect/meta/callback:`

Exchange code for short-lived token via:

GET `https://graph.facebook.com/v18.0/oauth/access_token`

Exchange short-lived for long-lived token (60-day) via:

GET `https://graph.facebook.com/v18.0/oauth/access_token?grant_type=fb_exchange_token&fb_exchange_token={short_token}`

Fetch ad accounts: GET `/me/adaccounts?fields=id,name,currency`

Save each `ad_account` record with encrypted long-lived token.

Trigger: `sync_meta_data.delay(ad_account_id)`

--- Data Sync Worker ---

@celery_app.task `sync_meta_data(ad_account_id):`

Load and decrypt access token.

Fetch campaigns: GET

`/ {account_id} /campaigns?fields=id,name,status,objective,daily_budget`

Fetch ad sets for each campaign: GET

`/ {campaign_id} /adsets?fields=id,name,status,daily_budget`

Fetch insights: GET `/ {account_id} /insights`

`?fields=spend,impressions,clicks,reach,actions,action_values,cpm,cpc,ctr&time_increment=1&date_preset=last_90d&level=adset`

For actions field: find `action_type=offsite_conversion.fb_pixel_purchase` for conversions.

Convert spend from platform currency to cents (multiply by 100).

Upsert campaigns, ad_sets, ad_insights (UNIQUE on `ad_set_id + date`).

--- Token Refresh ---

Check `token_expires_at` before each sync.

If expires within 7 days, re-fetch long-lived token and update DB.

```
If token invalid (error code 190), mark ad_account sync_status=token_expired.
Send notification to org admin.
```

```
--- Rate Limiting ---
```

```
Meta API returns X-Business-Use-Case-Usage header with usage %.
If usage > 75%, sleep for 60 seconds between batch requests.
On error code 17 (rate limit), raise RateLimitError, retry after 300s.
```

```
IDEMPOTENCY: all upserts on unique constraints.
```

YOU DO THIS MANUALLY — Google setup

1. In Google Cloud Console: enable Google Ads API. Create OAuth2 credentials (Web application type). Add `http://localhost:8000/api/connect/google/callback` as Authorised redirect URI.
2. Add Client ID and Client Secret to `.env` as `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`.
3. Add your Google Ads developer token to `.env` as `GOOGLE_ADS_DEVELOPER_TOKEN`.
4. Create a test Google Ads account (Google provides test accounts at `ads.google.com`).

CURSOR PROMPT — Google Ads OAuth + data ingestion

```
Build app/services/connectors/google.py and app/workers/sync_google.py.
```

```
--- OAuth Flow ---
```

```
Use google-auth-oauthlib library: pip install google-auth-oauthlib google-ads
```

```
GET /api/connect/google:
```

```
Generate auth URL with scopes: https://www.googleapis.com/auth/adwords
Store state nonce in Redis.
```

```
GET /api/connect/google/callback:
```

```
Exchange code for credentials (access_token + refresh_token).
```

```
Encrypt and store both tokens. refresh_token is permanent — store carefully.
```

```
Fetch accessible customers: GET
```

```
https://googleads.googleapis.com/v16/customers:listAccessibleCustomers
```

```
Save each as ad_account record.
```

```
Trigger: sync_google_data.delay(ad_account_id)
```

```
--- Data Sync Worker ---
```

```
@celery_app.task sync_google_data(ad_account_id):
```

```
Use google-ads Python client library.
```

```
Fetch campaigns using GAQL:
```

```
SELECT campaign.id, campaign.name, campaign.status,
campaign.bidding_strategy_type
```

```
FROM campaign WHERE campaign.status != REMOVED
```

```
Fetch ad groups:
```

```
SELECT ad_group.id, ad_group.name, ad_group.status, campaign.id
```

```
FROM ad_group WHERE ad_group.status != REMOVED
```

```
Fetch metrics (last 90 days):
```

```
SELECT ad_group.id, segments.date, metrics.cost_micros, metrics.impressions,
metrics.clicks, metrics.conversions, metrics.conversions_value
```

```
FROM ad_group_performance_view
```

```
WHERE segments.date DURING LAST_90_DAYS
```

```
Convert cost_micros to cents: cost_micros / 10000
```

```
Upsert campaigns (map Google ad_groups to our ad_sets), ad_insights.
```

```
--- Token Refresh ---
```

```
Google refresh tokens do not expire unless revoked.
```

```
Use google.oauth2.credentials.Credentials with refresh_token.  
On 401, refresh using refresh_token before retrying.
```

```
IDEMPOTENCY: all upserts on unique constraints.
```

☑ PHASE COMPLETE WHEN...

- ❑ Meta OAuth flow completes, ad accounts saved with encrypted tokens.
- ❑ Meta sync imports campaigns, ad sets, and 90 days of daily insights.
- ❑ Google OAuth flow completes, ad accounts saved.
- ❑ Google sync imports campaigns, ad groups (as ad_sets), and 90 days of metrics.
- ❑ Running either sync twice produces zero duplicates.
- ❑ Token refresh logic fires correctly when token is near expiry.
- ❑ Rate limit handling does not crash the worker — it retries gracefully.

PHASE 5 Attribution Engine — Tiers 1 to 4 **3–4 weeks**

This is the core intelligence of PFAM. Build each tier separately, test each independently, then wire them together in the orchestrator.

 Read this entire phase before writing any code. Understand all 4 tiers first. Then code them in order.

 CURSOR PROMPT — Attribution engine base structure

```
Create app/services/attribution/ package with:
__init__.py
orchestrator.py (main entry point – runs tiers in order)
tier1_click_id.py
tier2_conversion_id.py
tier3_sku_weighted.py
tier4_blended.py
models.py (AttributionResult dataclass)

In models.py, define:
@dataclass
class AttributionResult:
    order_id: str
    ad_set_id: str | None
    attribution_tier: int
    confidence_score: float
    attribution_method: str
    matched_click_id: str | None
    attributed_revenue_cents: int

In orchestrator.py, define:
def attribute_order(order: Order, ad_accounts: list, window_days: int = 7) -> AttributionResult:
    Try Tier 1. If result is not None, return it.
    Try Tier 2. If result is not None, return it.
    Try Tier 3. If result is not None, return it.
    Fall back to Tier 4. Return result.
    If no spend data at all, return AttributionResult with tier=0, confidence=0, ad_set_id=None.

The orchestrator should run for ALL unattributed orders after each sync cycle.
Already attributed orders (tier 1 or 2) should NOT be re-run unless force=True.
```

 CURSOR PROMPT — Tier 1 — Direct Click ID Match

```
Build app/services/attribution/tier1_click_id.py.

def tier1_attribute(order: Order, ad_insights: list, window_days: int) -> AttributionResult | None:
    Extract UTM params from order source (Shopify stores these in order.source_name,
    order.landing_site, or order.note_attributes).
    Look for these parameters:
    fbclid -> Meta click ID
```



```

gclid → Google click ID
ttclid → TikTok click ID
If fbclid found:
    Query ad_insights for Meta ad_sets that were active on order.created_at.
    Find the ad_set where fbclid was captured (if Meta Pixel event data
    available).
    If exact match found: return AttributionResult(tier=1, confidence=0.95,
    matched_click_id=fbclid)
    Same logic for gclid (Google) and ttclid (TikTok).
    If no click ID found in order data: return None.

IMPORTANT: Shopify UTM params are in landing_site URL as query params.
    Parse landing_site URL to extract fbclid/gclid.
    Also check note_attributes for any manually captured UTMs.

Test: Create test order with fbclid in landing_site. Verify tier=1,
confidence=0.95.

```

CURSOR PROMPT — Tier 2 — Conversion ID Match

```

Build app/services/attribution/tier2_conversion_id.py.

def tier2_attribute(order: Order, ad_accounts: list, window_days: int) ->
AttributionResult | None:
    Check if order has a matching Meta Pixel purchase event or Google Ads
    conversion.
    Meta: When Meta Pixel fires a Purchase event, it captures the order value and
    optionally a custom event ID. Match on order total and approximate timestamp.
    Query ad_insights where:
        platform = meta
        date = order.created_at.date()
        conversions > 0
    If single ad_set matches (only one running that day with that conversion
    count)
        and conversion_value within 5% of order total:
            return AttributionResult(tier=2, confidence=0.85)
    Google: Similar logic using Google Ads conversion data.
        Match on conversion_value_cents within 5% of order total.
        Match on date = order.created_at.date().
    If ambiguous (multiple candidates): return the one with highest spend on that
    day.
    If no match: return None.

Test: Create test order matching conversion value of a known campaign. Verify
tier=2.

```

CURSOR PROMPT — Tier 3 — SKU-Weighted Attribution

```

Build app/services/attribution/tier3_sku_weighted.py.

def tier3_attribute(order: Order, ad_insights: list, window_days: int) ->
AttributionResult | None:
    Get all SKUs in this order from line_items.
    Get all active campaigns in the attribution window (last window_days days).
    For each campaign/ad_set:
        Calculate: sku_overlap_score = (sum of order line_item revenue for SKUs
        that this campaign was promoting) / total order revenue

```

Note: We infer which SKUs a campaign promotes from historical order patterns.
 Use: for each ad_set, what SKUs appear most in attributed orders (Tier1/2 only)?

```
Build a sku_to_campaign_affinity dict from existing Tier1/2 data.
If any campaign has sku_overlap_score > 0:
    Distribute attribution proportionally to sku_overlap_score * ad_spend.
    winner = ad_set with highest (sku_overlap_score * spend).
    return AttributionResult(tier=3, confidence=0.70, ad_set_id=winner)
If no SKU affinity data (insufficient Tier1/2 history): return None.
```

NOTE: Tier 3 requires at least 30 Tier1/2 attributed orders to build affinity data.

If fewer than 30, skip to Tier 4.

CURSOR PROMPT — Tier 4 — Blended Proportional Attribution

Build app/services/attribution/tier4_blended.py.

```
def tier4_attribute(order: Order, ad_insights: list, window_days: int) ->
AttributionResult:
    Get all active ad_sets in the attribution window with spend > 0.
    Calculate total_spend_window = sum of all ad spend across all platforms in
    window.
    For each ad_set:
        weight = ad_set_spend_in_window / total_spend_window
    Winner = ad_set with highest weight (highest spend proportion).
    return AttributionResult(tier=4, confidence=0.50, ad_set_id=winner)
```

This is the fallback — always returns a result if ANY spend data exists.
 If total_spend_window = 0: return AttributionResult(tier=0, confidence=0,
 ad_set_id=None)

CURSOR PROMPT — Attribution orchestrator Celery task

Build app/workers/run_attribution.py.

```
@celery_app.task
def run_attribution_for_org(org_id: str, force_retier: bool = False):
    Load all orders for org where:
        - force_retier=False: only orders with no attributed_orders record
        - force_retier=True: all orders in last 90 days
    Load all ad_insights for the org (last 90 days).
    Load org attribution_window_days setting (default 7).
    For each order: run orchestrator.attribute_order(order, ...)
    Bulk upsert attributed_orders (UNIQUE on order_id + ad_set_id).
    Log: total orders, tier breakdown (how many tier1/2/3/4/unattributed), coverage
    %.
```

This task is triggered after every sync cycle completes.
 run_attribution_for_org.delay(org_id) called at end of sync_shopify_orders task.

PHASE COMPLETE WHEN...

- ☐ Tier 1: Order with fbclid in landing_site correctly attributed with tier=1, confidence=0.95.
- ☐ Tier 1: Order with gclid correctly attributed with tier=1.

- Tier 2: Order matching conversion value attributed with tier=2, confidence=0.85.
- Tier 3: With 30+ Tier1/2 orders, SKU-weighted attribution fires for remaining orders.
- Tier 4: All remaining orders get tier=4, confidence=0.50.
- Running attribution twice on same orders: zero duplicate attributed_orders records.
- Coverage % logged: what % of orders are attributed to at least one ad set.

PHASE 6 Profit Calculation Engine & Returns Modeling**🕒 2–3 weeks****CURSOR PROMPT** — SKU return rate computationBuild `app/services/profit/return_rates.py`.

```
def compute_sku_return_rates(org_id: str) -> None:
    For each SKU in the org:
        trailing_90d_rate = SUM(returns.quantity_returned WHERE refunded_at >= now()-90days)
                                / SUM(line_items.quantity WHERE ordered_at >= now()-90days)
        trailing_180d_rate = same but 180 days
    Upsert sku_return_rates table. If manual_override_rate is set, keep it – do not overwrite.
```

```
def get_return_rate_for_sku(org_id: str, sku: str) -> float:
    Load sku_return_rates for this sku.
    If manual_override_rate is set: return that.
    If trailing_180d_rate exists: return that.
    If trailing_90d_rate exists: return that.
    Else: return org-level average return rate (fallback).
```

Note: For orders < 45 days old, returns are "estimated" (expected but not yet received).

For orders >= 45 days old, use actual returns from returns table.
Flag this distinction in the profit calculation output.

CURSOR PROMPT — Core profit calculation engineBuild `app/services/profit/calculator.py`.

CRITICAL: Use Python Decimal for ALL intermediate calculations. Never float.
from decimal import Decimal, ROUND_HALF_UP

```
def calculate_profit_for_adset(
    org_id: str,
    ad_set_id: str,
    window_type: str, # daily, 7d, 14d, 30d
    window_start: datetime,
    window_end: datetime,
) -> ProfitMetric:

    # 1. ATTRIBUTED REVENUE
    orders = load attributed_orders WHERE ad_set_id=X AND window within range
    attributed_revenue_cents = SUM(order.total_amount_cents) for these orders

    # 2. AD SPEND
    spend_cents = SUM(ad_insights.spend_cents) WHERE ad_set_id=X AND date in window

    # 3. ATTRIBUTED COGS
    attributed_cogs_cents = 0
    For each order in attributed orders:
        For each line_item in order:
            cogs = get_cogs_for_sku(org_id, line_item.sku, line_item.unit_price_cents)
            attributed_cogs_cents += cogs * line_item.quantity
```

```

# 4. ESTIMATED RETURNS
estimated_returns_cents = 0
For each order in attributed orders:
    age_days = (now() - order.ordered_at).days
    if age_days >= 45:
        actual_returns = SUM(returns.refund_amount_cents WHERE order_id = order.id)
        estimated_returns_cents += actual_returns
    else:
        For each line_item:
            rate = get_return_rate_for_sku(org_id, line_item.sku)
            estimated_returns_cents += int(line_item.unit_price_cents *
line_item.quantity * rate)

# 5. PLATFORM FEES
fee_rate = get_platform_fee_rate(org_id) # default 0.029 (2.9%)
platform_fees_cents = int(attributed_revenue_cents * Decimal(str(fee_rate)))

# 6. NET PROFIT
net_profit_cents = (attributed_revenue_cents - spend_cents
    - attributed_cogs_cents - estimated_returns_cents - platform_fees_cents)

# 7. DERIVED METRICS
net_profit_pct = (net_profit_cents / attributed_revenue_cents * 100) if
attributed_revenue_cents > 0 else 0
true_roas = (attributed_revenue_cents / spend_cents) if spend_cents > 0 else 0

# 8. ATTRIBUTION COVERAGE
total_orders_in_window = count orders placed in window
attribution_coverage_pct = len(attributed_orders) / total_orders_in_window if
total_orders_in_window > 0 else 0

return ProfitMetric(all fields...)

def get_cogs_for_sku(org_id, sku, unit_price_cents) -> int:
    Check cogs_settings for exact SKU match (scope=sku, scope_value=sku).
    Check cogs_settings for category match.
    Check cogs_settings for global percentage.
    If cogs_type=absolute: return cogs_value_cents
    If cogs_type=percentage: return int(unit_price_cents * cogs_value / 100)
    If none found: return 0 (and flag as estimated/missing)

```

CURSOR PROMPT — Profit calculation Celery task

Build app/workers/calculate_profit.py.

```

@celery_app.task
def calculate_profit_for_org(org_id: str):
    Load all ad_sets for this org.
    For each ad_set, compute profit for windows: daily (last 90 days), 7d, 14d, 30d.
    For daily windows: calculate one ProfitMetric per day for last 90 days.
    For rolling windows (7d/14d/30d): calculate one ProfitMetric per window type.
    Upsert profit_metrics (UNIQUE on ad_set_id, window_type, window_start,
window_end).
    Set computed_at = now().

Trigger chain: sync → attribution → profit calculation.

```

```
In sync_shopify_orders task, at the end after attribution:  
  calculate_profit_for_org.delay(org_id)
```

IDEMPOTENCY: running twice must produce same profit_metrics records, not duplicates.

☑ PHASE COMPLETE WHEN...

- ❑ Profit calculated for 3+ ad sets with real data from Phase 3 and 4.
- ❑ Manual verification: pick one campaign. Calculate net profit by hand from raw data. PFAM result within $\pm 1\%$.
- ❑ Returns modeled: orders < 45 days use estimated rates; orders ≥ 45 days use actuals.
- ❑ COGS: works for absolute (cents) and percentage methods.
- ❑ Running calculate_profit_for_org twice: zero duplicate profit_metrics rows.
- ❑ net_profit_pct and true_roas computed correctly.

PHASE 7 Automation Rules Engine + Action Execution**🕒 2–3 weeks**

⚠️ This phase touches real ad accounts with real money. Test **EXCLUSIVELY** with test ad accounts until you are 100% confident in the logic. Never run against a real account until every checkpoint passes.

CURSOR PROMPT — Rules evaluation engine

```
Build app/services/rules/evaluator.py.

CONDITION SCHEMA (stored as JSONB in automation_rules.conditions):
[
  {"metric": "net_profit_cents", "operator": "lt", "value": 0, "window": "7d"},
  {"metric": "order_count", "operator": "gt", "value": 10, "window": "7d"}
]
logic: "AND" (or "OR")

Supported metrics: net_profit_cents, net_profit_pct, true_roas, spend_cents,
estimated_returns_cents, order_count, attribution_coverage_pct
Supported operators: lt, gt, lte, gte, eq
Supported windows: 1d, 3d, 7d, 14d, 30d

def evaluate_rule(rule: AutomationRule, org_id: str) -> list[RuleEvalResult]:
    Load all ad_sets in scope (filtered by rule.platform and rule.scope).
    For each ad_set:
        Load profit_metrics matching each condition window.
        Evaluate each condition against metric value.
        Apply AND/OR logic across all conditions.
        Check guardrails:
            min_orders: order_count in window >= rule.guardrails.min_orders (default 10)
            min_spend: spend_cents in window >= rule.guardrails.min_spend_cents (default
5000 = $50)
            max_actions_per_day: count audit_log entries for this account today <
rule.guardrails.max_daily_actions (default 10)
            idempotency: check audit_log - same rule + same entity + same window cannot
fire twice in 24 hours
        If ALL conditions pass AND ALL guardrails pass: append to triggered list.
        If conditions pass BUT guardrail blocks: append to blocked list.
    Return RuleEvalResult(triggered=[], blocked=[], evaluated_count=N)

def evaluate_all_rules_for_org(org_id: str):
    Load all automation_rules WHERE is_active=true AND org_id=org_id.
    For each rule: call evaluate_rule(rule, org_id).
    For triggered entities: queue action via execute_rule_action.delay().
    Upsert rule_executions record.
    Trigger notifications for triggered + blocked_by_guardrail.
```

CURSOR PROMPT — Action execution worker

```
Build app/workers/execute_action.py.

@celery_app.task(bind=True, max_retries=3)
def execute_rule_action(self, action_payload: dict):
    action_payload contains:
```

```

    {entity_type, entity_id, platform, action_type, action_params,
      rule_id, org_id, metric_snapshot, idempotency_key}

# IDEMPOTENCY CHECK
Check Redis: if idempotency_key already exists, log and return (already
executed).
Set Redis key with 25-hour TTL: redis.set(idempotency_key, executed, ex=90000)

# EXECUTE ACTION
Load and decrypt platform access token for org.
if action_type == "pause":
    if platform == "meta":
        POST https://graph.facebook.com/v18.0/{entity_id}
        body: {"status": "PAUSED"}
        headers: Authorization Bearer {access_token}
    elif platform == "google":
        Use Google Ads API to set campaign/ad group status to PAUSED.

elif action_type == "budget_decrease":
    Load current daily_budget from DB.
    new_budget = int(current_budget * (1 - action_params.decrease_pct/100))
    Enforce minimum budget floor (never reduce below $5/day).
    Update via platform API.

# AUDIT LOG - always write this, even on failure
INSERT into audit_log:
    actor_type=system, action_type=action_type, entity_id=entity_id,
    platform=platform, rule_id=rule_id, metric_snapshot=metric_snapshot,
    api_response_code=response.status_code
NOTE: audit_log is INSERT ONLY. Never update this record.

# ON FAILURE
On API error: raise self.retry(countdown=300*2**self.request.retries)
After max retries: write audit_log entry with api_response_code=0, trigger
failure notification.

# TRIGGER NOTIFICATION
send_notification.delay(org_id, trigger_type="rule_action", payload={...})

```

CURSOR PROMPT — Rules CRUD API endpoints

Build app/routers/rules.py:

```

GET /api/rules - list all rules for org (include last execution summary)
POST /api/rules - create rule. Validate conditions schema. Default
is_active=False.
PUT /api/rules/{id} - update rule. Cannot update while rule is evaluating.
DELETE /api/rules/{id} - only Owner/Admin role.
POST /api/rules/{id}/toggle - activate or deactivate. Require explicit body:
{"active": true}.
POST /api/rules/{id}/backtest - run evaluation against last 30 days WITHOUT
executing actions.
Return: {"would_have_fired": N_times, "affected_ad_sets": [...],
"estimated_impact": {...}}

```

All endpoints: verify org_id from JWT. Analyst role can create/edit. Read-Only cannot.


```
GET /api/audit-log – paginated, filterable by date/platform/action_type/entity_id.  
POST /api/audit-log/{action_id}/reverse – undo action (un-pause or reset budget).  
  This calls the platform API to reverse the action.  
  Writes a NEW audit_log entry for the reversal. Never edits the original.
```

☑ PHASE COMPLETE WHEN...

- ☐ Rule with condition "7d net_profit < 0 AND orders > 10" correctly evaluates against real data.
- ☐ Guardrail test: rule with min_orders=100 does NOT fire on ad set with 15 orders.
- ☐ Idempotency test: same rule fires once, not twice, in a 24-hour window.
- ☐ Audit log: every action creates an immutable record. No UPDATE on audit_log ever.
- ☐ Undo: reversing a pause calls the platform API and writes a new audit log entry.
- ☐ Backtest: returns historical would-have-fired data without executing anything.
- ☐ Max daily actions guardrail prevents runaway automation.

PHASE 8 COGS Management, Settings & RBAC**🕒 1–1.5 weeks****CURSOR PROMPT** — COGS management endpointsBuild `app/routers/cogs.py`:

```

GET /api/cogs - list all COGS settings for org
POST /api/cogs - create/update COGS entry.
  Validate: if cogs_type=absolute, cogs_value_cents must be positive integer.
  Validate: if cogs_type=percentage, value must be 0-100.
  Warn if cogs_value_cents > unit_price_cents for that SKU (possible data error).
POST /api/cogs/import - accept CSV upload.
  Expected columns: sku, cogs_value, currency
  Parse with pandas. Validate all rows before writing any.
  Upsert cogs_settings for each valid row.
  Return: {imported: N, skipped: M, errors: [...]}
POST /api/cogs/import-from-shopify - pull cost_per_item from all Shopify variants.
  Loop all line_items in DB. For SKUs with unit_cogs_source=shopify and
  unit_cogs_cents > 0:
    Create cogs_settings entries. Mark source=shopify.
POST /api/skus/{sku}/return-rate - manual override return rate for a SKU.

```

After any COGS update: `trigger calculate_profit_for_org.delay(org_id)`
 All historical `profit_metrics` will be recomputed with new COGS data.

CURSOR PROMPT — RBAC enforcement middlewareBuild `app/middleware/rbac.py`.

Define permission constants:

```

OWNER = ["all"]
ADMIN = ["view", "edit", "create_rules", "manage_team", "manage_connectors"]
ANALYST = ["view", "create_rules", "edit_rules"]
READ_ONLY = ["view"]

```

Create `require_role(*roles)` dependency for FastAPI:

```

def require_role(*allowed_roles):
    async def check(request: Request):
        user = load_user_from_db_using_clerk_user_id_from_jwt()
        if user.role not in allowed_roles and "owner" not in allowed_roles:
            raise HTTPException(403, detail={"code": "INSUFFICIENT_PERMISSIONS"})
    return Depends(check)

```

Apply to all endpoints:

```

Rules creation: require_role("owner","admin","analyst")
Connector disconnect: require_role("owner","admin")
Team management: require_role("owner","admin")
Billing: require_role("owner")
All GET endpoints: require_role("owner","admin","analyst","read_only")

```

Build `app/routers/team.py`:

```

GET /api/team - list users in org
POST /api/team/invite - send email invitation (create pending_invite record,
send via Resend)
PUT /api/team/{user_id}/role - change role (Owner only)

```

```
DELETE /api/team/{user_id} – remove user (invalidate sessions via Clerk API)
```

☑ PHASE COMPLETE WHEN...

- ❑ COGS CSV import works — valid rows imported, invalid rows reported without crashing.
- ❑ Shopify COGS import pulls cost_per_item for variants that have it.
- ❑ After COGS update, profit_metrics are recomputed within 5 minutes.
- ❑ READ_ONLY user: POST /api/rules returns 403.
- ❑ ANALYST user: POST /api/rules succeeds. DELETE /api/connectors/{id} returns 403.
- ❑ OWNER can do everything.

PHASE 9 Dashboard API Endpoints + Frontend UI**🕒 2–3 weeks****CURSOR PROMPT** — Dashboard API endpoints

```
Build app/routers/dashboard.py and app/routers/campaigns.py.

--- Overview ---
GET /api/dashboard/overview?start={date}&end={date}
  Return: {
    total_spend_cents, total_revenue_cents, total_cogs_cents,
    total_returns_cents, total_net_profit_cents, net_profit_pct, true_roas,
    campaigns_paused_today (from audit_log),
    platform_breakdown: [{platform, spend, revenue, net_profit, net_profit_pct}],
    last_sync_at per platform
  }

--- Campaigns ---
GET /api/campaigns?start={date}&end={date}&platform={} &status={}
  Return sorted list of campaigns with profit_metrics for requested date range.
  JOIN ad_sets → campaigns → profit_metrics.
  Include: attribution_coverage_pct, latest status from ad platform.

GET /api/campaigns/{id}/profit-breakdown
  Return stacked breakdown for one campaign:
  {revenue, spend, cogs, returns, fees, net_profit — all in cents and as % of revenue}

GET /api/adsets/{id}/attributed-orders?page=1&limit=50
  Return paginated attributed orders for this ad set.
  Include: order_id, ordered_at, total, sku_list, attribution_tier,
  confidence_score.

--- Products ---
GET /api/products/profitability
  Group line_items by SKU. For each SKU:
  {sku, product_name, units_sold, revenue_cents, cogs_cents, cogs_source,
  return_rate, return_cost_cents, net_margin_pct, campaign_count}

All queries filter by org_id. All monetary values in cents. Add Redis cache (5 min TTL).
```

CURSOR PROMPT — Build the dashboard UI — Account Overview

```
Build src/app/(dashboard)/overview/page.tsx.
Use shadcn/ui components and Recharts for charts.
Fetch data from GET /api/dashboard/overview using React Query.

Layout:
  Row 1: 4 KPI cards side by side.
    - Total Net Profit (color: green if positive, red if negative)
    - Total Ad Spend (gray)
    - Net Profit Margin % (semantic color)
    - Campaigns Paused Today (blue)
  Each KPI card: label (small gray text) + large bold value + delta vs prior period.
```

Row 2: Platform tiles (one per connected platform: Meta, Google).
Each tile: platform logo + spend + revenue + net_profit + profit% badge.

Row 3: Net Profit trend chart (Recharts LineChart).
X axis: dates. Y axis: net_profit_cents / 100.
Green line above zero, red line (or fill) below zero.
Reference line at y=0 labeled Break-even.

Row 4: Automation activity feed (last 5 actions from audit log).
Each item: action icon + campaign name + action + time ago + Undo button.

Bottom bar: Last synced times per platform. Sync Now button.

Loading state: skeleton placeholders for all cards and charts.
Empty state: if no platforms connected, show onboarding CTA.

CURSOR PROMPT — Build the dashboard UI — Campaigns Table

Build `src/app/(dashboard)/campaigns/page.tsx`.

Full-width data table using `shadcn/ui` Table component.

Columns:

- Status badge (green/amber/red based on net_profit_pct)
- Platform icon (Meta/Google logo)
- Campaign Name (bold, clickable to expand)
- Spend (\$)
- Revenue (\$)
- COGS (\$)
- Returns (\$)
- Net Profit (\$ - colored green/red)
- Net Profit % (colored, bold)
- True ROAS
- Attribution (dot + tier label)
- Status (Active/Paused chip)
- Actions (pause icon, rule icon)

All columns sortable. Default sort: net_profit_cents ascending (worst first).
Filter bar above table: platform pills (All/Meta/Google), status filter, date range.

Clicking a row opens a slide-in right panel with:

- Profit breakdown bar chart (horizontal stacked)
- Attribution inspector (tier breakdown %)
- 7/14/30-day trend chart

All monetary values displayed as dollars (divide cents by 100, format with \$).
Table has loading skeleton and empty state.

PHASE COMPLETE WHEN...

- ☐ Account overview loads with real data from your connected test accounts.
- ☐ KPI cards show correct totals matching manual DB query.
- ☐ Net profit trend chart renders correctly — green above zero, red fill below zero.
- ☐ Campaign table loads with all campaigns sorted worst-profit first.

- ❑ Clicking a campaign row opens the right panel with profit breakdown.
- ❑ All monetary values display as dollars, not cents.
- ❑ Empty state shows when no platforms connected.

PHASE 10 Notifications, Exports & Reporting**🕒 1–1.5 weeks****CURSOR PROMPT** — Notification system

```
Build app/services/notifications.py and app/workers/send_notification.py.

@celery_app.task
def send_notification(org_id: str, trigger_type: str, payload: dict):
    Load org notification settings (which channels are enabled for this
    trigger_type).
    For each enabled channel:

        EMAIL (via Resend):
            Build HTML email: campaign name, action taken, reason, metric values, Undo
            link.
            POST https://api.resend.com/emails
            headers: Authorization Bearer {RESEND_API_KEY}
            body: {from, to, subject, html}

        SLACK (via incoming webhook):
            POST to org.slack_webhook_url
            body: {"text": "PFAM: Paused *[campaign]* - 7d net profit: -${amount}",
                  "attachments": [{"text": "Undo this action", "actions": [...]}]}

        WEBHOOK:
            POST to org.webhook_url
            Sign payload with HMAC-SHA256 using org.webhook_secret.
            Header: X-PFAM-Signature: sha256={hex_digest}

    Insert notifications record with delivered_at and delivery_status.
    On failure: retry up to 3 times. After 3: mark status=failed.

Build app/routers/notifications.py:
GET /api/notifications - last 90 days, paginated
PUT /api/notifications/settings - update channel settings for org
```

CURSOR PROMPT — CSV and PDF export

```
Build app/services/exports.py.

def generate_campaign_csv(org_id: str, start: date, end: date) -> str:
    Query profit_metrics with campaign/ad_set join for date range.
    Build CSV using pandas DataFrame.
    Columns: Date, Platform, Campaign, Ad Set, Spend($), Revenue($), COGS($),
             Returns($), Fees($), Net Profit($), Net Profit%, True ROAS,
             Orders, Attribution Coverage%.
    Upload CSV to Cloudflare R2: exports/{org_id}/{filename}.csv
    Return pre-signed URL (valid 15 minutes).

def generate_summary_pdf(org_id: str, start: date, end: date) -> str:
    Use reportlab or weasyprint library.
    Page 1: header with org name + date range + 4 KPI tiles.
    Page 2: Net Profit trend chart (use matplotlib to render, embed as image).
    Page 3: Top 10 / Bottom 10 campaigns table.
    Page 4: Platform comparison table.
```

```
Upload to R2. Return pre-signed URL.
```

```
Build app/routers/reports.py:
```

```
GET /api/reports/export?format=csv&start={}&end={}
```

```
    Trigger export as background Celery task.
```

```
    Return job_id.
```

```
GET /api/reports/export/{job_id}/status
```

```
    Return {status: pending|complete|failed, download_url: ...}
```

```
Frontend: export button triggers job, polls status, opens download URL when ready.
```

☑ PHASE COMPLETE WHEN...

- ☐ Rule trigger sends email notification with correct campaign name, metric, and Undo link.
- ☐ Slack notification delivered when webhook URL configured.
- ☐ Webhook notification has valid HMAC-SHA256 signature.
- ☐ CSV export generates correctly — all columns present, values in dollars (not cents).
- ☐ Pre-signed R2 URL works for download and expires after 15 minutes.

PHASE 11 Stripe Billing & Subscription Management

 1 week

YOU DO THIS MANUALLY — Stripe setup

1. In Stripe dashboard (test mode): create Products for each plan — Starter (\$99), Growth (\$299), Pro (\$599), Scale (\$1299).
2. Create monthly recurring Prices for each product. Copy price IDs to .env.
3. Set up Stripe Customer Portal at dashboard.stripe.com/test/settings/billing/portal.
4. Add webhook endpoint: <https://your-railway-url/api/webhooks/stripe>. Select events: customer.subscription.created, customer.subscription.updated, customer.subscription.deleted, invoice.payment_failed, invoice.payment_succeeded.
5. Copy webhook signing secret to .env as STRIPE_WEBHOOK_SECRET.

CURSOR PROMPT — Stripe billing integration

```
Build app/routers/billing.py.

POST /api/billing/checkout — create Stripe Checkout session
  Load or create Stripe Customer for org (store stripe_customer_id in
  organizations).
  Create Checkout session with price_id from request body.
  Return {checkout_url} — frontend redirects user to this URL.

POST /api/billing/portal — create Stripe Customer Portal session
  Return {portal_url} — frontend redirects for plan changes, cancellation,
  invoices.
  Owner role only.

GET /api/billing/subscription — current subscription details
  Load from Stripe API using stripe_subscription_id.
  Return: {plan, status, current_period_end, monthly_ad_spend_cents}

POST /api/webhooks/stripe — handle Stripe webhook events (NO AUTH on this
endpoint)
  Verify webhook signature: stripe.Webhook.construct_event(payload, sig,
  STRIPE_WEBHOOK_SECRET)
  On customer.subscription.created/updated: update organizations.billing_plan.
  On invoice.payment_failed: set billing_status=past_due, send notification to
  owner.
  On customer.subscription.deleted: set billing_plan=free, restrict features.
  All webhook handling must be idempotent (Stripe can send same event twice).
  Use Stripe event ID: if already processed (check Redis), skip.

Frontend: src/app/(dashboard)/settings/billing/page.tsx
  Show current plan, usage meter (monthly ad spend vs plan limit), upgrade button,
  Manage Billing button (opens Stripe Portal), invoice history.
```

PHASE COMPLETE WHEN...

- ☐ Checkout flow: user clicks upgrade, completes Stripe test payment, plan updates in DB.
- ☐ Webhook: subscription.created event correctly updates organizations.billing_plan.
- ☐ Webhook idempotency: sending the same event twice does not create duplicate records.
- ☐ Customer Portal: user can change plan and cancel from the portal.

- ❑ Failed payment: triggers email notification to org owner.

PHASE 12 Automation UI, Onboarding Flow & Settings Pages

🕒 1.5–2 weeks

CURSOR PROMPT — Automation rules UI

Build `src/app/(dashboard)/automation/page.tsx`.

Two-panel layout:

Left: rules list table (name, platform, conditions summary, action, ON/OFF toggle, last triggered).

Right (slide-in on row click): rule detail – full conditions, execution history, edit/delete/backtest buttons.

Create Rule modal (triggered by "+ Create Rule" button):

Step 1 – Platform & Scope: dropdown for Meta/Google, scope (Campaign/Ad Set).

Step 2 – Condition Builder:

Sentence structure: "When [metric dropdown] [operator dropdown] [value input]"

" + Add Condition" button adds another row. Max 5 conditions.

AND / OR toggle between conditions.

Step 3 – Action: dropdown (Pause / Reduce budget X% / Increase budget X% / Alert only).

Step 4 – Guardrails: `min_orders` input (default 10), `min_spend` input (default \$50).

Step 5 – Notifications: checkboxes for Email / Slack / Webhook.

Preview button: calls `POST /api/rules/{id}/backtest`. Shows results before saving.

Save button: creates rule in Draft (OFF) state.

User must manually toggle ON to activate.

The ON/OFF toggle: large, clear, with confirmation dialog for first activation:

"This rule will automatically pause ad sets on your Meta account. Are you sure?"

CURSOR PROMPT — Onboarding wizard and settings pages

Build `src/app/(dashboard)/onboarding/page.tsx` – 5-step wizard.

Step 1: Connect Shopify (OAuth button → redirect to `/api/connect/shopify`)

Step 2: Connect Meta (OAuth button) + optional Google (OAuth button)

Step 3: Configure COGS (3 option cards: Import from Shopify / Upload CSV / Set % estimate)

Step 4: Create first rule (pre-filled template: "Pause if 7d profit < \$0 AND orders > 10")

Step 5: Done – confetti animation, link to dashboard.

Progress saved per step in DB. Resume from last incomplete step on next login.

"Skip for now" on each step – shows warning about reduced accuracy.

Build `src/app/(dashboard)/settings/` pages:

`settings/connectors/page.tsx`: list connected platforms, sync status, sync now, disconnect.

`settings/cogs/page.tsx`: COGS table, import buttons, per-SKU override.

`settings/team/page.tsx`: invite, role management, remove user.

`settings/notifications/page.tsx`: channel toggles, Slack webhook input, test send.

`settings/organization/page.tsx`: org name, currency, attribution window (7/14/30d).

☑ PHASE COMPLETE WHEN...

- ❑ Onboarding wizard completes all 5 steps with real OAuth connections.
- ❑ Progress persists on refresh — resuming at correct step.
- ❑ Rule builder: creating a rule with 2 conditions and a guardrail works end-to-end.
- ❑ Backtest shows historical would-have-fired results.
- ❑ Activating a rule requires explicit toggle + confirmation dialog.
- ❑ Settings pages: all CRUD operations work (COGS, team, notifications).

PHASE 13 Tier 5 — ML Attribution Model **4–6 weeks**

Do not start this phase until you have at least 500 Tier 1/2 attributed orders in your database. Those attributed orders are your training labels. Without real data, this model is worthless. Use Phase 1–12 in production with a real merchant first.

 YOU DO THIS MANUALLY — ML prerequisites

1. Check your DB: `SELECT COUNT(*) FROM attributed_orders WHERE attribution_tier IN (1,2).` Must be ≥ 500 .
2. Install ML libraries: `pip install scikit-learn xgboost mlflow shap pandas numpy`
3. Create a free MLflow tracking server or use `mlflow.set_tracking_uri("mlruns")` for local tracking.
4. Export training data: `SELECT ao.*, o.ordered_at, o.total_amount_cents, o.customer_id, li.sku, ai.spend_cents, ai.impressions FROM attributed_orders ao JOIN orders o ... WHERE ao.attribution_tier IN (1,2).` Save as CSV.

 CURSOR PROMPT — Build the ML attribution model — training pipeline

Build `app/services/attribution/tier5_ml.py` and `app/ml/train_model.py`.

--- FEATURE ENGINEERING ---

```
def build_features(order, ad_set, ad_insight) -> dict:
    return {
        "hour_of_day": order.ordered_at.hour,
        "day_of_week": order.ordered_at.weekday(),
        "days_since_campaign_start": (order.ordered_at - campaign.start_date).days,
        "order_value_cents": order.total_amount_cents,
        "order_value_vs_campaign_aov": order.total_amount_cents /
campaign_avg_order_value,
        "campaign_spend_in_window_cents": ad_insight.spend_cents,
        "campaign_impressions_in_window": ad_insight.impressions,
        "customer_is_returning": 1 if customer_has_prior_orders else 0,
        "sku_campaign_affinity_score": sku_affinity_for_campaign(sku, campaign_id),
        "platform_meta": 1 if platform == "meta" else 0,
        "platform_google": 1 if platform == "google" else 0,
    }
```

--- TRAINING ---

```
def train_attribution_model(org_id: str):
    Load all Tier 1/2 attributed_orders as positive examples (label = correct
ad_set_id).
    For each positive example, generate N negative examples
    (other ad sets active in same window = wrong answers).
    Build feature matrix X and binary label y (1=correct match, 0=wrong).
    Split: 80% train, 20% test.
    Train XGBoost classifier:
        model = xgb.XGBClassifier(n_estimators=200, max_depth=6, learning_rate=0.1,
scale_pos_weight=neg_count/pos_count, eval_metric=auc)
        model.fit(X_train, y_train, eval_set=[(X_test, y_test)])
    Evaluate: AUC-ROC on test set. Must be > 0.75 before deploying.
    Log experiment with MLflow: mlflow.log_metric("auc", auc_score)
    Save model: mlflow.xgboost.log_model(model, "attribution_model")
    Save to Cloudflare R2: models/{org_id}/attribution_model_v{version}.pkl
```

```

--- PREDICTION ---
def tier5_attribute(order, ad_sets_active, model) -> AttributionResult | None:
    For each active ad_set in attribution window:
        features = build_features(order, ad_set, ...)
        score = model.predict_proba([features])[0][1] # probability of correct match
        winner = ad_set with highest score
        confidence = winner_score (already 0-1)
        If winner_score < 0.55: return None (not confident enough, fall through to Tier 4)
    return AttributionResult(tier=5, confidence=confidence, ad_set_id=winner.id)

```

CURSOR PROMPT — Model training Celery task + model serving

Build `app/workers/train_ml_model.py`.

```

@celery_app.task
def retrain_attribution_model(org_id: str):
    Run train_attribution_model(org_id).
    If new model AUC > current production model AUC: promote to production.
    Update org settings: current_model_version, model_auc, last_trained_at.
    Log model metrics to audit_log (actor_type=system, action_type=model_retrain).

```

Schedule: run weekly via Celery Beat.

```

Add to celery_app.py: beat_schedule = {"retrain-weekly": {"task": "...",
"schedule": crontab(day_of_week=0, hour=2)}}

```

Model loading:

```

Load model at worker startup (cache in memory — do not reload per-request).
If model file not found in R2: fall through to Tier 4 (graceful degradation).
Reload model after retraining completes.

```

Update `orchestrator.py` to add Tier 5 between Tier 4:

```

After Tier 3 fails, try Tier 5 (if model loaded).
If Tier 5 confidence < 0.55, fall through to Tier 4.
Tier 5 before Tier 4 — ML is better than pure blended when model is confident.

```

PHASE COMPLETE WHEN...

- ☐ Training pipeline runs without errors on your real attribution data.
- ☐ AUC-ROC on test set is ≥ 0.75 before model is promoted.
- ☐ Tier 5 fires for orders where model confidence ≥ 0.55 .
- ☐ Orders where Tier 5 confidence < 0.55 correctly fall through to Tier 4.
- ☐ Model retrains weekly via Celery Beat. Old model replaced only if new AUC is better.
- ☐ If model file is missing from R2: attribution still works via Tiers 1–4.

PHASE 14 Production Deployment & Launch Checklist **3–5 days** **YOU DO THIS MANUALLY** — Deploy to production

1. Frontend → Vercel: connect pfam-frontend repo, set all NEXT_PUBLIC_ env vars, deploy.
2. Backend → Railway: create Web Service from pfam-backend repo. Set all env vars including DATABASE_URL (Neon main branch), ENCRYPTION_KEY, all platform API keys.
3. Workers → Railway: create Worker Service from same repo. Start command: celery -A celery_app worker --loglevel=info
4. Beat → Railway: create Cron Service. Start command: celery -A celery_app beat --loglevel=info
5. Update all OAuth redirect URIs (Shopify, Meta, Google) from localhost to your production URLs.
6. Set STRIPE_WEBHOOK_SECRET from the production Stripe webhook endpoint (not test).
7. Enable Sentry DSN for both frontend and backend projects.

Pre-launch check	Test	Pass?
Tenant isolation	Create 2 orgs. Confirm org A cannot see org B data via API.	☐
Money precision	Create order with \$99.99 value. Verify stored as 9999 cents.	☐
Audit log immutability	Attempt UPDATE on audit_log — must fail at DB level.	☐
Sync idempotency	Run sync twice. Zero duplicate orders/insights.	☐
Attribution coverage	With real ad account: coverage rate $\geq 60\%$ for Tier 1/2.	☐
Profit accuracy	Pick 3 campaigns. Manually calc profit. PFAM within $\pm 1\%$.	☐
Rules guardrail	Rule with min_orders=100 on ad set with 5 orders: must NOT fire.	☐
Max daily actions	Set max 3 actions/day. Trigger rule 5 times. Only 3 execute.	☐
Token encryption	Check DB: access_token_enc column is ciphertext, not plaintext.	☐
RBAC	Read-Only JWT on POST /api/rules: must return 403.	☐
Rate limits	Hammer Sync Now button rapidly: rate limit kicks in at attempt 2.	☐
Webhook signature	Verify HMAC-SHA256 signature on delivered webhook payload.	☐

Pre-launch check	Test	Pass?
Stripe	Complete test checkout. Subscription plan updates in DB.	☐
Email notification	Trigger a rule. Email arrives within 5 minutes.	☐
Export	Download CSV. All values in dollars. Open in Excel — no corruption.	☐

💡 Put the system under real load before your first paying customer. Use Locust or k6 to simulate 50 concurrent users hitting the dashboard. Fix any slow queries you find (add EXPLAIN ANALYZE in Neon).

Version	Date	Notes
1.0	February 2026	Complete A–Z implementation plan. 14 phases. Full SRS including 5-tier ML attribution. Solo builder + Cursor-ready.